

Amtrak

Nicole Mc Duffie and Theodore Sither

Universität Potsdam, Am Neuen Palais 10, 14469 Potsdam, Germany

Abstract. This paper discusses the various approaches our team explored to solve Flatland-generated environments using Answer Set Programming (ASP) in order model railway systems. As we started to develop our initial base solution, we encountered hardware compatibility challenges, requiring us to first upgrade our version of the Flatland repository to Python 3.8. Once we had a working base solution, we then extended Flatland’s capabilities to model station stops along train routes, modifying environment lp file generation scripts accordingly. We examined the impact of station stops on problem solvability, observing a correlation between increased stops and unsolvable environments. We investigated transforming Flatland into weighted graph representations (WGR). Our research showed that WGRs alone were insufficient for solving Flatland environments effectively. However, we discovered that using multi-shot solving techniques significantly improved the performance of weighted graph approaches. Additionally, we found that WGRs excel at calculating the shortest possible paths for agents within the environment, making them potentially valuable components in hybrid approaches or other WGR-based solutions.

1 Initial Implementation and Obstacles

1.1 Python 3.7 Incompatibility

Very early on, our team’s hardware had compatibility issues with the Flatland environment. Unlike older Macs that use Intel processors with an `x86_64` architecture, both of our teammates’ machines use Apple Silicon with `ARM64` architecture. Released in 2018, Python 3.7 was never built or maintained for `ARM64` macOS because it predated the 2020 release of Apple Silicon by a few Python versions [2]. There is therefore no precompiled version of Python 3.7 that runs natively on Apple Silicon including through Conda Forge, Anaconda’s package manager for Python, which never included Python 3.7 in its regular `ARM64` build process [10,3].

Ideally the Flatland-rl maintainers would upgrade their project to a newer version of Python, but considering the time and effort required for such an undertaking, developers have had to devise various methods to continue to use older, unsupported Python versions including those that have reached their end of life [6,3]). Apple offers Rosetta 2, an `x86_64` emulation layer, enabling the execution of Intel-based applications on ARM Macs [9]. While this allows for

the installation of Python 3.7, its performance is compromised due to on-the-fly translation. Alternatively, Python 3.7 can be run within a Docker container that simulates an `x86_64` environment ensuring strict compatibility, but the associated additional layer of abstraction to our development process exceeded our desired level of technical complexity [7].

The necessary modifications ¹ mainly focused on updating the `pip requirements.txt` file with package dependency versions compatible with Python 3.8. We also had to adjust the installation source for Clingo, switching from Potassco’s channel to Conda Forge, as well as a few bug fixes in the code base itself. Changes to this setup process are reflected in our repository’s `README.md` ².

Although this upgrade resolved our compatibility issues, it introduced a new challenge: debugging unexpected behavior when trying to fit the Flatland environment to our implementation. Whenever we encountered an error—whether it was a direct exception or simply unexpected behavior—it was sometimes unclear whether the issue stemmed from our code and logic or from subtle differences in Python versions. This added complexity to troubleshooting sometimes delayed our development, but overall the move to Python 3.8 was sufficiently sustainable, allowing us to avoid resorting to alternative, more technically demanding approaches.

1.2 Base Solution

While working on our initial base solution we wanted to automate the process of decoding the transition bitmaps from each track type’s ID numbers. We created an embedded Python function within our clingo solution to translate these track IDs into the 16 bit representation of possible movements. Although embedded Python functions are normally a particularly handy way to incorporate the flexibility of Python into the powerful but limited functionality of clingo code, it was only upon completion automatization that we realized Flatland cannot run Python functions from within clingo code. Our consolation was that this code was still useful in avoiding having to manually transcribe these transitions, we simply modified it to be a stand alone Python script ³ that then generates the same information in a separate `lp` file to be included in our implementation ⁴.

Although there are many valid ways of transcribing this information, the structure we chose for our initial `base.lp` solution ⁵ took the form of multiple `track_option(Track,M,D)` atoms per `Track` (track IDs) where `M` denoted the four possible movements of the train and `D` represented cardinal directions. From these environment-independent representations of the different track types, the

¹ Available as a patch: <https://github.com/tsither/amtrak/blob/main/0001-adjust-main-flatland-repo-to-work-with-python3.8-for.patch>

² <https://github.com/tsither/amtrak/blob/main/amtrak/README.md>

³ https://github.com/tsither/amtrak/blob/main/amtrak/utils/generate_track_options.py

⁴ https://github.com/tsither/amtrak/blob/main/amtrak/track_options.lp

⁵ <https://github.com/tsither/amtrak/blob/main/amtrak/base.lp>

specific layout of an environment’s grid could be translated into all possible agent actions of each cell: `option((X1,Y1),(X2,Y2),D1,M,D2)` represented the relationship between two cells with coordinates $(X1,Y1)$ and $(X2,Y2)$, along with the agent’s starting direction $D1$ at the first cell and the agent’s direction $D2$ after reaching the second cell. This structure made `action` atom generation a simple matter of randomly selecting one of the `option` atoms with a starting cell matching the agent’s current cell.

Our solution also included `at(ID,(X,Y),T,D)` atoms to keep track of the agents’ positions at each time step. A chain of `actions` and `ats` building off of each other represented each agents’ path from its starting cell to destination cell. The requirement for each agent to achieve a `done` atom by reaching its end destination ensured complete paths withing the given time frame and served as the basis for our solution’s optimization. By maximizing for these `done` atoms, the solution is prioritizing solutions where agents spend the most time steps at their end cells and therefore have the shortest paths.

1.3 Status Generation in paths.csv

Another challenge we encountered was consistently achieving an accurate visualization of our solution using Flatland. When testing our solution on Flatland generated environments using the Flatland solver we discovered the last agent to reach its end destination would never quite achieve a `done` status in the generated `paths.csv` file. This agent would similarly disappear from the map right before reaching its end cell, despite our clingo output verifying that our agents were indeed reaching their final destinations in our implementation, as depicted in Table 1.

Time	paths.csv Output	Clingo Output
27	0;27;(17, 13);w;moving;move_forward	action(train(0),move_forward,27) at(0,(17,13),27,w)
	1;27;(11, 14);e;moving;move_forward	action(train(1),move_forward,27) at(1,(11,14),27,e)
	0;28;None;w;done;wait	action(train(0),wait,28) at(0,(17,12),28,w)
28	1;28;(11, 15);e;moving;move_forward	action(train(1),move_forward,28) at(1,(11,15),28,e)
	0;29;None;w;done;wait	action(train(0),wait,29) at(0,(17,12),29,w)
29	1;29;(11, 22);e;moving;move_forward	action(train(1),move_forward,29) at(1,(11,22),29,e)
		action(train(0),wait,30) at(0,(17,12),30,w)
30		action(train(1),wait,30) at(1,(11,23),30,e)
		at(0,(17,12),31,w)
31		at(1,(11,23),31,e)

Table 1: Example `paths.csv` contents with the accompanying clingo output depicting this `done` issue. In this example the first 26 time steps are truncated and agents had the following end facts: `end(0,(17,12),31).` `end(1,(11,23),44).`

We then dedicated significant time to attempting to comprehend the precise mechanisms by which these statuses were generated. In what would become a recurring theme throughout our project, the interplay between Kep’s Flatland repository and the original Flatland-rl repository it is built around proved to be highly challenging to navigate, resulting in an opaque understanding of Flatland’s translation logic. Our advice to future students would be to start at-

tempting to understand the relationship between these two code bases as early in the project as possible, as it will certainly help development along the way.

When combing through the repositories did not yield insights into the mismatch between our solution and the Flatland logic, we instead started to test various adjustments in our code, hypothesizing that the issue is likely related to an off-by-one error (sometimes referred to as a "fencepost error"). We attempted to shift the frame of time steps in various ways to test its effect on status generation, especially the **done** and **ready_to_depart** statuses.

The variations we attempted were all based on larger questions around how to define and conceptualize each of the given atoms and their elements. For example, does an **action** atom represent the movement of the agent from its current cell or the movement that brought the agent to its current cell? If a agent is at its current cell at time T , should its next **action** be at time T or $T+1$? Should agents only be allowed **actions** during the period between its start and end times or should those time steps be filled with wait **actions**? If a agent has a starting time of T , should its first **action** be at time T , $T+1$, or even $T-1$ and similarly should that first **action** bring the agent to the starting cell (if so from where?) or have it depart from that cell? All of these questions have implications on how the time steps could be handled in a solution and there is not necessarily any one correct approach. We experimented with these different variations to try to understand what logic Flatland was expecting as input, but ultimately we were not able to completely address this issue.

The **ready_to_depart** status introduced additional complications as it did not consistently align with the agents' starting time step and shifting the overall time step frame, disrupting the entire file. The **ready_to_depart** status would usually appear on the lines associated with the agent's starting time step, but in some cases, like in the case of `envs/lp/env_002-2_2.lp`, where the starting time is zero for both agents, the **ready_to_depart** status is generated in the line for time step one. This shift in frame can be seen throughout the rest of the file, where agents' positions continued to be one ahead of those in our clingo output and the file terminates one time step before agents reach their end cell, preventing either from achieving a **done** status. We tried many of the same adjustments to attempt to solve this problem (while maintaining our solution's ability to solve other environments) including more brute force attempts like hard coding a `action(train(ID),wait,-1)` for each agent despite -1 not technically being a valid time step, all of which failed to correct this frame shift.

Ultimately the configuration that came the closest to generating the appropriate statuses included adding **wait actions** to the time steps before the agent's start time and after the agent had reached its destination until its end time (time steps which had previously no **actions**).

In this conceptualization, **actions** represented the movement of an agent from its cell position at the same time step to the next cell position one time step in the future. To properly register the appropriate **ready_to_depart** status and therefore set the correct time step frame within `paths.csv`, the agent's first movement **action** would occur at its starting time step.

For example, for a agent defined as

```
train(0). start(0,(10,23),1,w).
```

The first few lines of the appropriate clingo output would be:

```
action(train(0),wait,0).
action(train(0),move_forward,1).  at(0,(10,23),2,w).
action(train(0),move_forward,2).  at(0,(10,22),3,w).
...
```

In most cases only the last agent failed to achieve a `done` status, but in a few cases like the `envs/lp/env_002-2_2.lp` example above where the `ready_to_depart` statuses were already on the wrong timestep, the agents came very close before `paths.csv` cut them off. In our clingo output these agents did indeed reach their end destinations with plenty of time steps to spare. We were assured that as long as we were confident all agents were truly completing their routes, we could put this issue aside to proceed with the more important aspects of our project. In the end this issue was not unique to our project, as other groups also mentioned experiencing it in class.

2 Modeling Multiple Station Stops

Looking beyond our initial solution, we wanted to explore the ways in which we could make Flatland more realistic in its representation of real life railway systems. Among the ideas we considered were modeling agents with varying priority in the case of breakdowns or number of passengers and agents of varying lengths instead of the single train car visualized in Flatland. Ultimately, we decided to introduce multiple station stops along an agent’s route, rather than solely a starting and ending point. We thought this was an approachable and interesting aspect of real railway systems that could be simulated. Our objective was to implement this feature in both our ASP solution but also add features to Kep’s Flatland repository in terms of environment building and visualization.

We successfully created logic to handle station stops in our ASP solution ⁶ as well as modified the `build.py` script to create environment `lp` files with the number of stops specified under our added `num_of_stops` parameter in the `envs/params.py` file. Unfortunately reflecting these station stops in the `pkl` and `png` files was beyond the scope of this project. We had hoped this feature could be accomplished with only modifications to Kep’s Flatland repository, but soon realized it would require changes to the much larger, denser Flatland-rl repository. Initial research into its available customizations followed by attempts in creating a modified version that could then be incorporated with Kep’s Flatland were made, but considering our limited time, we instead focused on the ASP side of this feature.

⁶ https://github.com/tsither/amtrak/blob/main/amtrak/station_stops.lp

Unsurprisingly, more stops per agent increased the likelihood of generating an unsolvable environment especially with smaller environment grids. We developed a script ⁷ to first build various environments of varying dimensions with different numbers of station stops per agent. Our solution was then run on each environment in order to tally the number of environments that resulted in an "UNSATISFIABLE", a timeout, or a viable model, the results of which are found in Table 2. Across all three grid sizes the increase in stops seems to be correlated with a decrease in an environments' solvability, a unsurprising pattern considering the complexity an out of the way stop can add to a agent's path.

		Num of Stops				
		0	1	2	3	4
40x40	1	21	41	38	43	Unsat
	0	9	8	12	7	Timeouts
	49	20	1	0	0	Solved
50x50	0	24	39	44	39	Unsat
	2	11	10	6	10	Timeouts
	48	15	1	0	1	Solved
60x60	0	30	34	32	38	Unsat
	1	11	14	15	12	Timeouts
	49	9	2	3	0	Solved

Table 2: To better understand the effect of number of stops on different environment grid dimensions (left), 50 environments were created for each combination of dimensions and number of stops. Our solution was then run on each environment to count up the number of unsolvable environments (Unsat), environments that resulted in a timeout (Timeouts), and environments that were able to generate a viable model.

Timeout duration was determined by one of the environment's dimensions multiplied by three (for example the 40 x 40 grids received 120 seconds) and when timeouts occurred, the lp file was moved to a separate folder so the reason for its timeout could be further investigated. Determining the fraction of these environments that are actually solvable or not would exceed the time available for this project. They do seem to occur far more frequently when stops are present, but not necessarily proportionally to the number of stops.

⁷ https://github.com/tsither/amtrak/blob/main/amtrak/utls/test_stops.py

3 Graph Representations in Flatland

Flatland is a discrete, 2 dimensional grid-based representation of a train transportation scheduling problem. The effectiveness of modeling transportation systems as discrete networks or graphs is well understood [1]. Answer set programming (ASP) positions itself as an effective tool for modeling and solving discrete combinatorial problems. In contrast to Flatland’s original intention as a reinforcement learning problem, ASP instead treats Flatland as a combinatorial problem. Graphs and networks have long been central components in the field of combinatorics [5]. Therefore, the motivation for abstracting the Flatland environment to a graph is to leverage the combinatorial nature of graphs while using the ASP engine.

3.1 Weighted Graph Representation

There are a variety of types of graph that can be used to accurately represent the Flatland environment. In this paper, we pursue a weighted graph representation (WGR). The following is how we abstract Flatland instances to WGRs:

Vertices / Nodes : Vertices⁸ are any cells that have track types that contain a switch of some kind. A choice of action can (or must, depending on the incoming direction) be made on these types of tracks⁹. The nodes also include all cells in which a train starts or ends.

Edges : Edges are abstract connections between nodes that contain a length or ‘weight’ representing how many ‘non-wait’ actions are needed to traverse from node to node. Cells that host track types that contain no switch options can be part of an edge. Those cells also must NOT be host to any start or end locations for any agents. Adjacent nodes with no cells in between have an edge length of one.

The Flatland environment can be understood as a second-order undirected graph. In standard undirected graphs (also known as first-order graphs), traversability—possible movements between nodes—is determined simply by identifying which nodes are connected by each edge. In contrast, navigating a graph representation of Flatland requires knowledge of the incoming direction to each node in order to determine which outgoing edges, and nodes thereby, are traversable.

The weighted graph approach was originally intended to be more efficient and less computationally demanding than a standard non-WGR approach¹⁰. The upside of a WGR is that it is not strictly necessary to define time steps for each agent on every cell, a common scaling issue with ASP in Flatland¹¹. Agent

⁸ Vertices and nodes are used interchangeably.

⁹ Complete list of track types and categorization can be found in appendix.

¹⁰ We do not claim this implementation is the optimal WGR of Flatland. Therefore these findings necessarily extend to all WGR approaches.

¹¹ This may be impossible, due to the requirement to define `action` atoms for every timestep. This requirement might force all timestep to be grounded in the program regardless.

movement and goal adherence can be orchestrated by knowing the time steps of agents on the nodes and the lengths of the edges between them. Therefore, the frameworks discussed in this paper pursue WGR methods that solve larger Flatland environments faster.

Hard Constraints We also propose the following three ‘hard constraints’. These are rules that give both necessary context to our approach and define correct agent behavior in the Flatland environment:

1. Agents must reach their destinations, and reach them on time.
2. Collisions between agents are forbidden, on nodes or edges.
3. Agents can only ‘wait’ on edges, never on nodes.

Fig. 1: Hard Constraints in WGRs

Constraints 1 and 2 ensure that the answer sets of a WGR program are solutions to the Flatland environment (along with the correct definition of **action** atoms downstream). Removing constraints 1 or 2 implies that NOT all of the WGR’s answer sets are potential Flatland solutions. Constraint 3 represents an intentional bias in our WGRs, intended to reduce the search space without overly reducing environment solvability.

3.2 Weighted Graph Representation - Basic

Our WGR Flatland solution takes the following three inputs:

1. *Flatland Instance* - Facts containing the cells, tracks, & agent information
2. *Graph* - Facts & rules that build a weighted graph representation of the *Flatland instance*
3. *Solve* - Rules that define correct movements of the agents (from the *Flatland instance*) on the WGR (built from the *Graph*)

Fig. 2: Input to an ASP graph-based Flatland environment

The basic WGR approach (WGR-B) conforms to typical clingo protocol: *Flatland-instance*, *Graph*, and *Solve* are all grounded and solved simultaneously. The output of this program is any number of answer sets that show the ‘actions’ of the agents throughout the graph. The logic within *Solve* tracks agent’s position in time on nodes only. Actions on nodes are selected from the `next_possible_node` atom:

```
next_possible_node(at(Current_Node), (IN_Current,OUT_Current,IN_Next),
Action, next(Next_Node), dist(Next_Length)).
```

`next_possible_node` represents the complete second-order graph for the given *Flatland-instance*.

Unfortunately, the performance of the WGR-B was poor. Simply put, this approach was unable to scale to even small environments (environments on the order of 30-40 sized grids with 2-10 agents).

The core issue arises from the enforcement of the hard constraints defined in Figure 1. In particular, constraint 2, ensuring that there be no collisions on edges led to the exponential increase in program run time, rendering this implementation useless when it came to solving larger Flatland environments.

Enforcing the collision constraint necessarily requires a time dimension component and a comparison between multiple agents. Every time a train enters a node, this constraint checks the time span for every other train using this edge, ensuring that the time spans do not overlap:

```

1 edge_collision :- ID1 != ID2,
2   at_node(train(ID1), NodeA, IN_A_1, DepTime1),
3   at_node(train(ID1), NodeB, IN_B_1, ArrTime1),
4
5   at_node(train(ID2), NodeB, IN_B_2, DepTime2),
6   at_node(train(ID2), NodeA, IN_A_2, ArrTime2),
7
8   node_action(train(ID1), NodeA, IN_A_1, Action1),
9   node_action(train(ID2), NodeB, IN_B_2, Action2),
10
11   next_possible_node(at(NodeA), (IN_A_1,_,_), Action1, next(NodeB), dist(Dist)),
12   next_possible_node(at(NodeB), (IN_B_2,_,_), Action2, next(NodeA), dist(Dist)),
13
14   DepTime1 < ArrTime2,
15   DepTime2 < ArrTime1.
```

Fig. 3: Hard Constraint 2 - Clingo syntax defining agent collisions on edges.

Enforcing this constraint led to an explosion in the grounding. Grounding is the conversion of a logic program with variables to an equivalent program without variables, a necessary and fundamental step in generating possible solutions or answer sets. In naive grounding, this translation to a variable-free program populates every variable with every constant that appears in the program. Although the grounder underlying ASP has a few tricks to ground rules more efficiently, this step can still be computationally expensive [8].

In the case of constraint 2, we suggest that the size of the constraint (the number of different variables) leads to the program's intractability. That is, there are 15 different variables that need to be populated with all the possible constants in the program. These constants are collected from elsewhere in the program as well; for example, the various possible time step or edge length integers. However, the main advantage of WGR is that grounding every time step is not necessary and even large environments should not have many unique edge lengths. So where were all of these constants coming from?

The Edge Building Problem After further exploration, we were forced to consider the process of how the edges themselves are constructed. The way

our weighted edges are defined is an iterative and recursive process using the `edge_step` rule:

```

1 edge_step((N,OUT), B, C, IN, L) :-
2   edge_step((N,OUT), A, B, _, L1),      % N=start node; A,B,C = intermediate cells
3   link(B, C, (_,IN)),                  % adjacent cells
4   not edge_step((N,OUT), C, B, _, _),
5   not node(B,_),
6   L=L1+1,                               % adjust length
7   L <= CC,                               % hard stop
8   B!=C,
9   cell_count(CC).                       % count of cells with tracks

```

Fig. 4: The recursive and iterative rule that serves as the precursor to `edge` atoms.

This rule counts non-node cells from node to node incrementally. Even though there is a condition defining that the next counted cell must not be a node (line 5), this rule will nevertheless ground all integers from 1 to the hard stopping condition in line 7, defined by the fact `cell_count`. In other words, if a 40x40 Flatland environment has a `cell_count(100)`, all integers 1 - 100 will be grounded. These integers combinatorially populate every variable in the constraint in Figure 3 during grounding. This inefficiency is at least partially responsible for the intractability of WGR-B.

3.3 Weighted Graph Representation - Iterative Solving

The iterative approach to the weighted graph representation (WGR-I) separates the original implementation (WGR-B) into two distinct parts. This method is intended to mitigate the ‘Edge Building Problem’ by leveraging the unidirectional dependency that *Solve* has on *Flatland-Instance* and *Graph*. In other words, the rules that define a solution on a WGR require a graph be defined. However, the converse is not true: a WGR does not require rules defining agent movements in order to build a weighted graph.

WGR-I first grounds and solves the *Flatland-Instance* and *Graph*. This builds the ‘graph’ of the *Flatland-Instance*. This second-order graph representation is then passed to *Solve* as facts via `next_possible_node`.

The WGR-I works by embedding a Python script inside *Solve*. Clingo’s python library builds a virtual Clingo engine within the script. This engine then inputs *Flatland-Instance* and *Graph* only. They are then grounded and solved in this isolated environment. We can then extract the `next_possible_node` atoms from that environment and pass them to *Solve* as facts.

This process is related to the concept of multi-shot solving. Multi-shot solving is typically for transient programs that contain state changes and involves the control or modification of different rules or constraints over time to handle these state changes[4]. Although WGR-I does not need to consider any changing states in the Flatland environment, the iterative solving process seemed to be applicable when it came to building the graph.

Ultimately, this approach also disappointed when it came to solving the actual Flatland environment. Enforcing the constraint forbidding collisions proved intractable once again. Grounding and solving the graph separately from the solution couldn't mitigate the grounding explosion this constraint caused¹². However, removing this constraint from the program entirely gave some interesting results.

Efficient Graph Building The motivation for a multi-step grounding and solving process was to avoid inefficient grounding arising from the interaction of the graph building and solving processes. If we remove the edge collision constraint from *Solve*, we can examine how this multi-step graph building in WGR-I compares to the simultaneous grounding and solving in WGR-B¹³. Figure 5 demonstrates that WGR-I allowed for faster grounding and solving times compared to the standard WGR-B, particularly as environments scaled. In fact, WGR-I outperforms WGR-B in every instance by a considerable margin. It is again important to emphasize that the underlying Clingo code is functionally identical in the different WGR methods. The only difference is the multi-step nature of WGR-I. The resulting answers sets are identical. The performance difference is especially noticeable in larger environments, as can be seen from Figure 5c where WGR-I can execute while WGR-B is forced to timeout.

Method	n agents	Time (s)
WGR-B	4	1.121
WGR-I	4	0.856
WGR-B	10	4.445
WGR-I	10	1.380

(a) 40x40 grid

Method	n agents	Time (s)
WGR-B	4	45.941
WGR-I	4	25.029
WGR-B	10	58.273
WGR-I	10	28.557

(b) 60x60 grid

Method	n agents	Time (s)
WGR-B	4	timeout
WGR-I	4	171.388
WGR-B	10	timeout
WGR-I	10	208.571

(c) 80x80 grid

Fig. 5: Solve times of WGR on different grid sizes; timeout set fixed 300 seconds. Edge collision constraint not enforced.

Even though the WGR-I is not optimized to provide valid Flatland solutions, this multi-step solving method could be invaluable in other kinds of ASP-based problems involving weighted graphs. We implore future ASP-Flatlanders pursuing weighted graph representations to utilize this framework as a simple, yet effective way of improving performance and reducing computational overhead.

¹² This issue requires further exploration, perhaps from future ASP-Flatlanders.

¹³ Answer sets now represent possible agent movements that may or may not contain edge collisions.

3.4 Weighted Graph Representation - Pathing

Pathing refers defining possible paths of agents from their start nodes to end destinations. A path is the tuple of nodes that an agent traverses such that they end on their ending destination.

The pathing approach to the weighted graph representation (WGR-P) begins by computing all of the possible paths from **start** and **end** cell for all agents. We utilize the multi-step solving method from WGR-I during this process for more efficient solving. The logic is similar to generating the **edge_step** atoms in the rule 4. However, instead of tracking edge lengths using adjacent cells, we track the total distance from an agents start position to end destination via valid edges between nodes:

```

1 path_step(ID, INDEX, (S,F), IN_Next, (B, C), TL) :-
2   end(ID,_,ET),
3   not TL>=ET,                                     % Hard stop
4   path_step(ID, PREV_IDX, (S,F), IN_Current, (A, B), L1),
5   next_possible_node(at(B), (IN_Current,_,IN_Next), _, next(C), (dist(L2))),
6   INDEX=PREV_IDX+1,                               % Count nodes
7   F!=B,
8   TL=L1+L2.
```

Fig. 6: The recursive and iterative rule that serves as the precursor to **path** atoms. The hard stopping condition ensures that the distance is less than the latest arrival time, avoiding infinite paths.

Using the rule in Figure 6, it is simple to then define a **path** atom that contains all possible paths and their distances for each agent from their starting location to ending destination.

Shortest Possible Path Once all possible paths are defined in our program, it is relatively straightforward in ASP to define the shortest possible path:

```
shortest_path_length(ID,S,L) :- path(ID,S,_,L), L = #min { Length : path(ID,_,_,Length) }.
```

Fig. 7: The shortest path distance for each agent can be defined in a single line of Clingo code.

We then define a **node_action** atom that, as a condition, says that the arrival time must be less than or equal to the shortest path length for that agent:

```

1 {node_action(train(ID), Current_Node, Action, (IN_Current,OUT_Current,IN_Next), Next_Node,
2   (dep(Current_Time),dist(Next_Length), arr(Current_Time + Next_Length)))} :-
3   ...
4   start(ID,_,Start_Time,_),
5   shortest_path_length(ID,_,Shortest),
6   Current_Time + Next_Length <= Shortest + Start_Time, % Ensures shortest path
7   ...
```

Fig. 8: The shortest possible path for each agent. Part of rule is removed due to spacing limit.

The rule in Figure 8 functionally defines the shortest possible path for each agent to their end destination. This is where the weighted graph approach might have considerable advantage over more standard approaches. It is likely possible to organize a similar rule without the use of a weighted graph. However, computing this kind of rule may be faster and more efficient using a compact and lightweight weighted graph representation, in conjunction with the multi-step solving framework.

Computing the shortest paths does not run into the same grounding issues as the different WGR solutions. In fact, often times computing the shortest possible paths is quite computationally approachable, as can be seen in Figure 9. Surprisingly, the solve times do not seem to increase linearly as the number of agents increases. In Table 9b the solve time is virtually equivalent in the same environment, regardless of the number of agents. This is a considerable strength of this approach, and can also exhibited in Table 9a. We speculate this is a result of the trains often using a limited number of started and ending locations, reducing the total number of constants that need to be ground.

Size	n agents	Time (s)
30x30	4	1.681
30x30	10	1.734
60x60	4	22.85
60x60	10	25.119
80x80	4	173.713
80x80	10	176.339

(a) Shortest path on varying environments

n agents	Time (s)
4	1.821
50	1.98

(b) 50x50 grid

Fig. 9: Solve times of computing the shortest possible paths.

3.5 WGR - Limitations

Modeling Correct Agent Behavior The WGR implementations presented in this paper, on their own, were unable to solve Flatland environments at scale. The cause is understood to be the enforcement of correct agent movement in a weighted graph framework. More specifically, enforcing constraint 2 - forbidding edge collisions on nodes - was intractable in large environments. This was due to the interaction between the large number of variables in that rule (figure 3 and the many integer constants that are grounded in the program generally. It should be noted that other implementation strategies were attempted to mitigate this issue, such as including more information in the `node_action` atom:

```
node_action(train(ID), Node, Action, (IN, OUT, IN_Next),
Next_Node, (dep(Dep_Time), dist(Dist), arr(Arr_Time))).
```

Fig. 10: Other WGR strategies to define agent movement.

Even though this rule allows for a smaller, more compact definition of the edge collision constraint, the grounding of this rule itself raised the same issues as grounding the previous edge collisions rule: too many variables. In all of the WGR approaches introduced in this paper, enforcing all constraints required a rule containing many variables. This was strictly necessary in order to handle the different dynamics of multi-agent network system. Therefore, the question might be raised: is enforcing correct agent behavior in weighted graphs a universal limitation for all WGR-ASP approaches in Flatland?

There are two obvious directions to tackle this problem. The first is to explore more efficient methods to require fewer variables in the rules of these constraints. The fewer variables defined in the rules, the faster a program might be. The other approach is to minimize the number of integer constants that are introduced before grounding. Some potential bottlenecks could be mitigated using the multi-step WGR-I approach (e.g. cell (x,y) format or iterative edge building). However, the following limitation makes it unclear as to if this bottom-up approach of reducing integer grounding is even possible...

Action Time Steps The last step for a WGR to become a Flatland solution- as described by the original Flatland library- is to define `action` atoms for all agents. These must adhere to the format: `action(ID,Action,Time)`. This unfortunately implies that all time steps must be ground regardless of implementation. This severely limits the main advantage that WGRs have in the first place: not all time steps need to be grounded to describe agent movement. This should not render WGR-only approaches immediately useless. For example, we speculate that one could send already solved `node_action` atoms to a separate solver as facts using WGR-I, and populate `actions` atoms along the edges avoiding the need for time steps to interact the the definition of `node_action`. However, regardless of the details of approach, this issue should be a crucial consideration in any future WGR implementations.

Acknowledgments. A big thank you to Kep for all the help and enthusiasm throughout the semester. We are also grateful to the group MadMotion¹⁴ for their ASP compatible user interface. Their application was definitely a time saver when constructing test environments and evaluating solution outputs.

References

1. Derrible, S., Kennedy, C.: Applications of graph theory and network science to transit network design. *Transport Reviews* **31**(4), 495–519 (2011). <https://doi.org/10.1080/01441647.2010.543709>
2. Foundation, P.S.: python.org. <https://www.python.org/downloads/release/python-379/> (2020), [Accessed 04-03-2025]
3. Foundation, P.S.: Status of Python versions — devguide.python.org. <https://devguide.python.org/versions/#versions> (2025), [Accessed 07-03-2025]

¹⁴ <https://github.com/M4X-VOGEL/Clingonia>

4. GEBSER, M., KAMINSKI, R., KAUFMANN, B., SCHAUB, T.: Multi-shot asp solving with clingo. *Theory and Practice of Logic Programming* **19**(1), 27–82 (2019). <https://doi.org/10.1017/S1471068418000054>
5. Harary, F.: *Graph theory* (on Demand Printing of 02787). CRC Press (2018)
6. Inc., A.: macos monterey 12.3 release notes. https://developer.apple.com/documentation/macos-release-notes/macos-12_3-release-notes (2025), [Accessed 04-03-2025]
7. tech kamar: How to Deploy specific version of Python using Docker — techkamar.medium.com. <https://techkamar.medium.com/how-to-deploy-specific-version-of-python-using-docker-96d387c16779> (2024), [Accessed 04-03-2025]
8. Kaufmann, B., Leone, N., Perri, S., Schaub, T.: Grounding and solving in answer set programming. *AI magazine* **37**(3), 25–32 (2016)
9. Rinik, V.: How to install Python under Rosetta 2 — medium.com. <https://medium.com/thinknum/how-to-install-python-under-rosetta-2-f98c0865e012> (2021), [Accessed 04-03-2025]
10. Wint, M.D.: Python 3.7 on Apple Silicon — tech.gorilla.co. <https://tech.gorilla.co/python-3-7-on-apple-silicon-de6a97c7f4d> (2022), [Accessed 07-03-2025]

A Appendix

A.1 WGR

A.2 Future directions

Iterative / Multi-step solving A WGR alone may not be a great candidate as a solution to Flatland. The edge collision grounding issue might be an obstacle facing other WGRs as well. However, these limitations might be avoided by leveraging the strengths of WGRs. Because WGRs have distinct conceptual components of the instance, graph, and any other operations applied on top, we believe that multi-step solving could be an incredibly effective tool if wielded correctly. The inherent nature of WGRs also afford other strengths and possibilities as well.

Shortest Possible Paths Utilizing shortest possible paths could be a promising avenue for future approaches for a few different reasons. Requiring agents to only traverse the nodes in their shortest path set is not a Flatland solution on its own due to the possibility of collisions on edges or nodes. However, by fixing which nodes agents are allowed to visit, a solution would need to only be defined along the time dimension. One could format a multi-step approach in which a weighted graph is computed and sent to the shortest path solver as facts. Then, the output from the shortest path solver be sent to the *Solve* file as facts. This might mitigate the grounding explosion caused by the edge collision constraint?

This rule could also be extended to hybrid standard and graph-based Flatland approaches. One might achieve this by defining rules requiring the agents to traverse nodes in the shortest possible path set, along with some other constraints

minimizing extraneous movement (e.g. forbidding agents to travel to nodes that are not in their shortest path set).

In real-world transportation networks, distances carry costs in the form of time taken and fuel expenditure. Reducing the total distance covered is almost certainly a fundamental part of a transportation system’s objective function. Requiring Flatland agents to adhere to their shortest possible path could align the Flatland environment closer to real transportation systems.

Subgraph A subgraph of the original WGR could be constructed given the shortest paths for each agent. This could be achieved by defining new edges and transitions using only the shortest possible paths for each agent. By definition, a solution exists in this subgraph. That is, all end nodes for each agent are reachable. A different kind of hybrid approach could be built if the edges were translated back into their original cell representations. A standard non-graph based solution could then be solved from scratch on this subgraph, using the shortest path cells as guides or not, the advantage here being that the environment’s search space has been reduced.

Track Types - Vertices and Edges The following track types comprise our edge components (assuming they don’t contain an agent start or end position) in the WGRs:



Fig. 11: Default edge track types in WGR

The following track types are always classified as vertices:



Fig. 12: Default vertex track types in WGR

Formal Definition The WGRs implemented in this paper can be formalized as the following six tuple:

- Graph** $G = \{D, T, V, \delta, E, w\}$
- Directions:** $D = \{n, e, s, w\}$
- Track Types:** $T = \{t_1, t_2, \dots, t_n\}$ and $O \subset T$ that contains tracks of type 1, tracks that contain no switches
- Vertices:** $V = \{v_i = (x, y) \mid x, y \in \mathbb{N}\}$ where: $v_i \in$ the set of start cells or end cells or $\exists t_i \in C$ where $C = \{cell((x, y), t_i) \mid t_i \notin O\}$
- Transition Function:** $\delta : V \times D \times T \rightarrow \mathcal{P}(D)$, where $\delta(v, d_{in}, t)$ gives the set of possible outgoing directions from vertex v when entered from direction d_{in} .
- Edges:** $E = \{(v_i, (d_{in}, d_{out}, d'_{in}), v_j, w) \mid v_i, v_j \in V, d_{in}, d_{out}, d'_{in} \in D\}$ where $d_{out} = \delta(v_i, d_{in}, t)$
- Weights** $w : E \rightarrow \mathbb{N}$ where $w((v_i, (d_{in}, d_{out}, d'_{in}), v_j))$ represents the length between v_i and v_j